

DevMap

VS Code Extension — Implementation Plan

A step-by-step guide to build a JS/Node/Express
topic tracker that learns as you vibe-code

For: Full Stack AI Engineers learning Node.js
Stack: TypeScript · VS Code API · Babel AST · Chart.js

What is DevMap?

DevMap is a VS Code extension that silently watches your JavaScript, Node.js, and Express projects. Every time you save a file, it parses your code into an Abstract Syntax Tree (AST), matches the syntax patterns against a built-in curriculum map, and shows you exactly which topics you have covered — in both a sidebar panel and a full web dashboard.

Component	Technology	What you learn
-----------	------------	----------------

File watcher	VS Code workspace API	Event-driven Node patterns
AST parser	@babel/parser or acorn	How JS is really structured
Topic matcher	Tree traversal in JS	Recursion, maps, sets
Sidebar panel	VS Code TreeView API	Provider pattern
Web dashboard	WebviewPanel + Chart.js	postMessage, HTML in Node
Curriculum map	topics.json	JS/Node/Express scope

Total build time estimate: 3–5 days for a working v1. Each phase is a standalone milestone — you can stop at any phase and already have something useful.

Phase 1 Scaffold the Extension

~2 hours · pure Node + VS Code setup

Goal

Get a working VS Code extension that activates, registers a command, and shows a notification. This is your Hello World — don't skip it.

Steps

Install tools	Run: <code>npm install -g yo generator-code</code> — this gives you the official VS Code extension generator.
Scaffold project	Run: <code>yo code</code> → choose TypeScript → name it devmap. This creates <code>package.json</code> , <code>extension.ts</code> , <code>tsconfig.json</code> .
Understand <code>package.json</code>	Look at 'contributes' (commands, views) and 'activationEvents'. This file IS your extension's manifest.
Register a command	In <code>extension.ts</code> , use <code>vscode.commands.registerCommand('devmap.scan', ...)</code> and test it via F5 in VS Code.
Add your first view	Add a 'contributes.viewsContainers' and 'contributes.views' entry in <code>package.json</code> for the sidebar slot.

Key files after Phase 1

- **package.json** — manifest, commands, view registrations
- **src/extension.ts** — `activate()` entry point
- **tsconfig.json** — TypeScript config (already generated)

Phase 2

File Watcher + AST Parser

~1 day · the core engine

Goal

Every time you save a .js or .ts file, parse it into an AST and walk the tree to find which syntax patterns are present. This is the brain of DevMap.

Steps

Install parser	npm install @babel/parser @babel/traverse — gives you a full JS AST with every node type labelled.
Hook onDidSave	In extension.ts: vscode.workspace.onDidSaveTextDocument(doc => scanFile(doc.uri.fsPath))
Parse to AST	Call babelParser.parse(code, {sourceType:'module', plugins:['typescript']}) — returns a full AST object.
Walk the tree	Use @babel/traverse to visit node types. Each visitor function receives a node. Log them first to understand the shape.
Scan workspace	On activation, also run vscode.workspace.findFiles('**/*.js,ts') to scan all existing files, not just on-save ones.
Return findings	Return a Set of matched topic keys (e.g. 'async_await', 'arrow_fn') from each file scan.

Example AST node matches:

AST Node Type	Topic detected
ArrowFunctionExpression	Arrow functions
AwaitExpression	Async / Await
ImportDeclaration	ES Modules (import/export)
ClassDeclaration	ES6 Classes
SpreadElement	Spread / Rest operator
TemplateLiteral	Template literals
CallExpression (require)	CommonJS modules
TryStatement	Error handling (try/catch)

Phase 3

Curriculum Map (topics.json)

~3 hours · define what you want to learn

Goal

Build a structured JSON file that defines every topic across three tracks. This file is the source of truth — the AST matcher reads from it, the sidebar renders from it, and the dashboard visualises it.

topics.json structure:

```
{ "tracks": [{ "id": "javascript", "label": "JavaScript Core",  
  "topics": [{ "id": "arrow_fn", "label": "Arrow functions",  
    "astPattern": "ArrowFunctionExpression", "docsUrl": "..."}] } ] }
```

JavaScript Core topics to include:

Variables & scope	Closures	Prototypes
Arrow functions	Destructuring	Spread / Rest
Template literals	Promises	Async / Await
ES Modules	Iterators	Generators
Classes	Error handling	Array methods

Node.js topics to include:

fs module	path module	Streams
EventEmitter	http module	process object
child_process	Buffers	env variables
CommonJS (require)	npm scripts	Debugging

Express topics to include:

App setup	Routing	Middleware
Error middleware	Request object	Response object
REST patterns	Static files	Body parsing
Auth middleware	Router module	CORS

Phase 4

Sidebar TreeView Panel

~half day · VS Code provider pattern

Goal

Render your topic progress as a collapsible tree in the VS Code sidebar. Each track is a top-level node; topics inside show a checkmark (covered) or a circle (not yet). Clicking a topic opens its MDN/Node docs page.

Steps

Create TopicProvider	Implement <code>vscode.TreeDataProvider</code> . Two methods required: <code>getTreeItem()</code> and <code>getChildren()</code> .
TopicItem class	Extend <code>vscode.TreeItem</code> . Set <code>iconPath</code> to a checkmark or circle based on covered status. Add a command for click-to-docs.
Register the provider	<code>vscode.window.registerTreeDataProvider('devmap.topicsView', new TopicProvider(store))</code>
Wire to state store	When the AST scanner updates covered topics, call <code>provider._onDidChangeTreeData.fire()</code> to refresh the sidebar.
Add status bar item	Show '✓ 12/45 topics' in the VS Code status bar using <code>vscode.window.createStatusBarItem</code> — always visible at a glance.

Phase 5

Web Dashboard (WebviewPanel)

~half day · HTML inside VS Code

Goal

A rich HTML dashboard that opens in a VS Code tab. Shows progress bars per track, a topic grid with covered/pending states, and a Chart.js bar chart. Data flows from your extension to the page via `postMessage`.

Steps

Create WebviewPanel	<code>vscode.window.createWebviewPanel('devmap', 'DevMap', vscode.ViewColumn.One, {enableScripts:true})</code>
Send data via message	<code>panel.webview.postMessage({ type:'update', data: store.getState() })</code> — call this after every scan.
Receive in HTML	<code>window.addEventListener('message', e => { if(e.data.type==='update') renderDashboard(e.data.data) })</code>
Build the UI	Pure HTML/CSS/JS in a template string. Use Chart.js (loaded via CDN) for the progress bar chart per track.
Topic grid	Render a CSS grid of topic pills — green background for covered, gray for pending. Click opens docs URL.
Add a reset button	A 'Clear progress' button posts a message back to extension: <code>panel.webview.onDidReceiveMessage(msg => ...)</code>

Tip: The webview is sandboxed. You cannot use Node APIs inside it — treat it like a browser page. All data must arrive via `postMessage` from the extension side.

Project Folder Structure

```
devmap/  
├── src/  
│   ├── extension.ts ← activate(), commands, watcher  
│   ├── scanner.ts ← AST parser + topic matcher  
│   ├── topicProvider.ts ← TreeDataProvider for sidebar  
│   ├── dashboardPanel.ts ← WebviewPanel + postMessage  
│   └── store.ts ← covered topics state (Map)  
├── data/  
│   └── topics.json ← curriculum map (3 tracks)  
├── media/  
│   ├── check.svg ← covered icon  
│   └── circle.svg ← pending icon  
├── package.json ← extension manifest  
└── tsconfig.json
```

What You Actually Learn by Building This

Phase	Node/JS Concept	Where it shows up
1	npm, package.json, modules	Extension scaffold and dependencies
1	TypeScript basics	All source files use .ts
2	Async/await, Promises	File reading, workspace scan APIs
2	Recursion, tree traversal	Walking the AST with @babel/traverse
2	Sets and Maps	Storing covered topics without duplicates
2	File system (fs module)	Reading source files to parse
3	JSON schema design	topics.json curriculum structure
4	Provider/Observer pattern	TreeDataProvider + onDidChangeTreeData
4	Event emitters	VS Code's EventEmitter internally
5	postMessage / events	Extension ↔ Webview communication
5	DOM manipulation	Dashboard HTML rendering
5	Sandboxed environments	Webview CSP and restrictions

Quick Start Commands

Install generator

```
npm install -g yo generator-code
```

Scaffold extension

```
yo code
```

Install dependencies

```
npm install @babel/parser @babel/traverse
```

Install TS types

```
npm install --save-dev @types/vscode @types/node
```

Run in debug mode

```
Press F5 in VS Code – opens Extension Dev Host
```

Watch + compile

```
npm run watch (generated in scaffold)
```

Package extension

```
npm install -g @vscode/vsce then vsce package
```

V2 Ideas — After You Ship V1

- Add an AI explainer — when you click a covered topic, call Claude API to explain what you used it for in your own code
- GitHub sync — push your progress JSON to a GitHub Gist so it persists across machines
- Streak tracking — show a daily heatmap of how many new topics you covered each day
- Quiz mode — the extension asks you a question about a topic you recently used
- Multi-language support — extend topics.json to cover Python, React, or SQL
- Team leaderboard — compare progress with teammates via a shared backend

Built for AI engineers learning full stack Node.js through real projects.